Final Project Report

Tic-Tac-Toe Game

Group Number: 2

Group Members:

Sajir Rashdi Karim 2621130042

Irfan Sadik Ahmed 2625435042

Ishteaque Ahnaf 2624254042

Md. Mahfuz Hayder Dilshad 2624833042

1. Introduction:

Tic-Tac-Toe is a classic two-player strategy game played on a 3x3 grid, widely used as an introductory project in programming courses because it exercises core procedural programming concepts: control flow, arrays, input validation, and modular function design. This report documents the design, implementation, and evaluation of a console-based Tic-Tac-Toe game written in the C programming language.

The objective of this project was to build a fully playable, two-player Tic-Tac-Toe game that runs entirely in a terminal/console environment, without any graphical interface or external libraries. The game needed to correctly handle turn-taking, input validation, win detection across all possible winning combinations, and draw detection, while presenting a clear and readable board to the players after every move.

1.1 Motivation

Beyond fulfilling the assignment requirement, this project served as a practical exercise in translating an abstract set of game rules into a working, robust program. Handling edge cases such as invalid input, out-of-range moves, and already-occupied slots required careful defensive programming, which reinforced core C programming skills including array indexing, conditional logic, and loop control.

1.2 Scope

The scope of this project is limited to a two-human-player, single-session console game. It does not include an AI opponent, networked multiplayer, persistent score tracking across sessions, or a graphical user interface. These are discussed further as potential directions in the Future Work section of this report.

2. Problem Statement:

The core problem this project addresses is: how can a simple, rule-based two-player game be implemented reliably in a low-level, procedural language like C, using only the standard library, while ensuring the program never crashes or enters an invalid state regardless of what the user types?

This breaks down into several concrete sub-problems that the implementation had to solve:

- Representing the 3x3 board in a way that is simple to update, display, and check for a winner.
- Validating player input so that non-numeric input, out-of-range slot numbers, and already-taken slots are all rejected gracefully without crashing the program.
- Correctly detecting a win across all eight possible winning lines: three rows, three columns, and two diagonals.
- Detecting a draw when the board fills up with no winner.
- Presenting the board clearly after every move so both players can track the game state.

3. System Design and Approach:

The program follows a straightforward procedural design, organized into three logical layers: configuration/state, UI rendering, and core game logic, tied together by a single main game loop. This separation keeps each function focused on a single responsibility, which makes the code easier to read, test, and extend.

3.1 Board Representation

The board is represented as a 10-element character array, board[10], where indices 1 through 9 correspond to the nine playable slots, and index 0 is left unused. Each slot is initialized to hold its own slot number as a character ('1' through '9'), which serves two purposes at once: it lets the board display slot numbers to guide the player's input before any moves are made, and it lets the program later distinguish an empty slot from an occupied one, since occupied slots are overwritten with 'X' or 'O'.

3.2 Turn Management

Two global variables, current_player and current_marker, track whose turn it is and which marker ('X' or 'O') they use. After every valid move, these are swapped using simple conditional (ternary) expressions, alternating the turn between Player 1 (X) and Player 2 (O).

3.3 Win and Draw Detection

Win detection is implemented by explicitly checking all eight possible winning lines: three horizontal rows, three vertical columns, and two diagonals. For each line, the function compares the three relevant board positions; if all three match (and are not the default numeric placeholders), a win is declared. A draw is detected separately by tracking a total_turns counter; if nine moves have been made without a winner being found, the board is full and the game ends in a tie.

4. Implementation Details:

The program is implemented as a single C source file, main.c, using only the standard stdio.h and stdlib.h libraries. Below is a walkthrough of the key components.

4.1 Constants and Global State

Two macros define the player markers, and global variables track the board and current turn:

```
#define PLAYER_X 'X'
#define PLAYER_O 'O'

char board[10] = {'0','1','2','3','4','5','6','7','8','9'};
char current_marker;
int current_player;
```

4.2 Rendering the Board

The draw_board() function clears the console using system("cls") and prints the current state of the 3x3 grid using the board array, with horizontal dividers between rows for visual clarity. Note that system("cls") is a Windows-specific command; running this program on Linux or macOS would require substituting "clear" instead for the screen-clearing behavior to work as intended.

4.3 Placing a Marker

The place_marker(int slot) function validates the requested slot before allowing a move. It rejects the move (returning 0) if the slot number is outside the 1-9 range, or if the slot has already been claimed by either PLAYER_X or PLAYER_O. Otherwise, it writes the current player's marker into the board array and returns 1 to indicate success:

```
int place_marker(int slot) {
    if (slot < 1 || slot > 9 ||
        board[slot] == PLAYER_X ||
        board[slot] == PLAYER_O) {
        return 0; // Invalid move
    }
    board[slot] = current_marker;
    return 1; // Valid move
}
```

4.4 Checking for a Winner

The check_winner() function checks all eight winning combinations by direct index comparison. This is a simple, readable approach that is easy to trace since each of the eight winning lines is checked in its own line of code, at the cost of some repetition compared to a more generalized loop-based approach.

4.5 The Main Game Loop

The main() function initializes the game state, then loops continuously while game_status remains 0 (ongoing). On each iteration, it draws the board, prompts the current player for a slot number, and validates the input in two stages: first confirming that scanf() successfully read an integer, and second passing the value to place_marker() to confirm the slot itself is valid. Invalid input at either stage prompts the player to try again without ending the game. After a successful move, the program increments the turn counter, checks for a win, then checks for a draw (nine turns reached with no winner), and finally swaps the active player if the game is still ongoing.

5. Testing and Results:

The program was tested manually by playing through multiple full games, covering each of the possible ways a game can end, as well as a range of invalid inputs, to confirm the program handles them gracefully.

5.1 Test Cases

- Row win: verified for all three rows (top, middle, bottom).
- Column win: verified for all three columns (left, middle, right).
- Diagonal win: verified for both diagonals (top-left to bottom-right, and top-right to bottom-left).
- Draw condition: played a full game to nine moves with no winner, confirming the tie message displays correctly.
- Invalid slot number: entered values below 1 and above 9, confirming the program rejects them and re-prompts.
- Occupied slot: attempted to place a marker on an already-taken slot, confirming the program rejects the move.
- Non-numeric input: entered letters instead of numbers, confirming the input buffer is cleared and the player is re-prompted without crashing.

5.2 Results

All test cases passed as expected. The program correctly identifies winners across every possible winning line, correctly detects draws, and does not crash or misbehave under any of the invalid input scenarios tested. The board renders clearly after each move, and turn order alternates correctly between the two players throughout every test game.

6. Challenges Faced:

One of the main challenges was ensuring robust input validation. Reading input with scanf("%d", &choice) can leave invalid characters in the input buffer if the user types a non-numeric value, which, if not handled, would cause the program to loop endlessly re-reading the same bad input. This was addressed by explicitly clearing the input buffer with a while (getchar() != '\n') loop whenever scanf() fails to read an integer.

Another consideration was keeping the win-checking logic both correct and readable. While a loop-driven approach checking rows, columns, and diagonals programmatically would reduce the number of lines of code, the explicit line-by-line comparison used here

was chosen for clarity, making it straightforward to verify by inspection that all eight winning combinations are covered.

7. Future Work:

Several extensions could improve and expand this project beyond its current scope:

- Single-player mode against a computer opponent, potentially using the minimax algorithm for optimal play.
- Persistent score tracking across multiple rounds within a session, and optionally saved to a file between sessions.
- Cross-platform screen clearing, replacing the Windows-specific system("cls") call with a portable solution.
- A graphical or web-based interface as an alternative to the console-based version.
- Refactoring win detection into a loop-driven check over a table of winning index combinations, reducing code repetition.

8. Conclusion:

This project successfully implements a complete, robust, two-player Tic-Tac-Toe game in C, meeting all core requirements: correct turn management, comprehensive win detection across all eight winning lines, draw detection, and defensive input validation that prevents invalid input from crashing or corrupting the game state. The modular structure, separating board configuration, UI rendering, and core game logic, keeps the codebase easy to read, test, and extend for future enhancements such as an AI opponent or persistent score tracking.

Through this project, the team gained practical experience with array-based state representation, defensive input handling, and systematic condition-checking logic in C, all of which are foundational skills applicable to a wide range of future programming projects.